САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа 1: CSS Flexbox, CSS Grid

Выполнил:

Якшин Артемий

K3440

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Пройти три интерактивные игры:

1. Flexbox Froggy (flexboxfroggy.com): расположение элементов средствами CSS Flexbox;
2. CSS Grid Garden (cssgridgarden.com): расположение элементов средствами CSS Grid;
3. Learn Git Branching (learngitbranching.js.org): основы работы с git. Нужно выполнить первые 4 урока и все уроки по удалённым репозиториям.

Собрать отчёт по шаблону, экспортировать его в PDF и открыть Pull Request в репозиторий курса.

Ход работы

1. Flexbox Froggy

В игре нужно расставить лягушек по кувшинкам своего цвета. Для этого пишется CSS для flex-контейнера `#pond` (у него уже задано `display: flex`), а на части уровней и для отдельных flex-элементов. Каждый уровень вводит новое свойство:

- `justify-content` выравнивает элементы вдоль главной оси (`flex-start`, `flex-end`, `center`, `space-between`, `space-around`);
- `align-items` выравнивает элементы вдоль поперечной оси;
- `flex-direction` задаёт направление главной оси (`row`, `row-reverse`, `column`, `column-reverse`). При `column` оси меняются ролями: `justify-content` работает по вертикали, а `align-items` по горизонтали. При `row-reverse` начало оси оказывается справа, поэтому `flex-end` прижимает элементы влево (уровень 10);
- `order` и `align-self` задаются отдельному элементу: они меняют его порядок и выравнивание, не трогая остальные элементы;
- `flex-wrap` разрешает перенос элементов на новые строки, а `flex-flow` объединяет `flex-direction` и `flex-wrap` в одно свойство;
- `align-content` распределяет строки по поперечной оси. Оно действует, только если элементы переносятся на несколько строк.

Все 24 уровня пройдены. Решения приведены в таблице 1.

Таблица 1. Решения уровней Flexbox Froggy

№	Свойство	Селектор	Решение
1	justify-content	`#pond`	`justify-content: flex-end;`
2	justify-content	`#pond`	`justify-content: center;`
3	justify-content	`#pond`	`justify-content: space-around;`
4	justify-content	`#pond`	`justify-content: space-between;`
5	align-items	`#pond`	`align-items: flex-end;`
6	align-items	`#pond`	`justify-content: center;` `align-items: center;`

7	align-items	#pond	justify-content: space-around; align-items: flex-end;
8	flex-direction	#pond	flex-direction: row-reverse;
9	flex-direction	#pond	flex-direction: column;
10	flex-direction	#pond	flex-direction: row-reverse; justify-content: flex-end;
11	flex-direction	#pond	flex-direction: column; justify-content: flex-end;
12	flex-direction	#pond	flex-direction: column-reverse; justify-content: space-between;
13	flex-direction	#pond	flex-direction: row-reverse; justify-content: center; align-items: flex-end;
14	order	.yellow	order: 1;
15	order	.red	order: -1;
16	align-self	.yellow	align-self: flex-end;
17	align-self	.yellow	order: 1; align-self: flex-end;
18	flex-wrap	#pond	flex-wrap: wrap;
19	flex-wrap	#pond	flex-direction: column; flex-wrap: wrap;
20	flex-flow	#pond	flex-flow: column wrap;
21	align-content	#pond	align-content: flex-start;
22	align-content	#pond	align-content: flex-end;
23	align-content	#pond	flex-direction: column-reverse; align-content: center;
24	align-content	#pond	flex-direction: column-reverse; flex-wrap: wrap-reverse; justify-content: center; align-content: space-between;

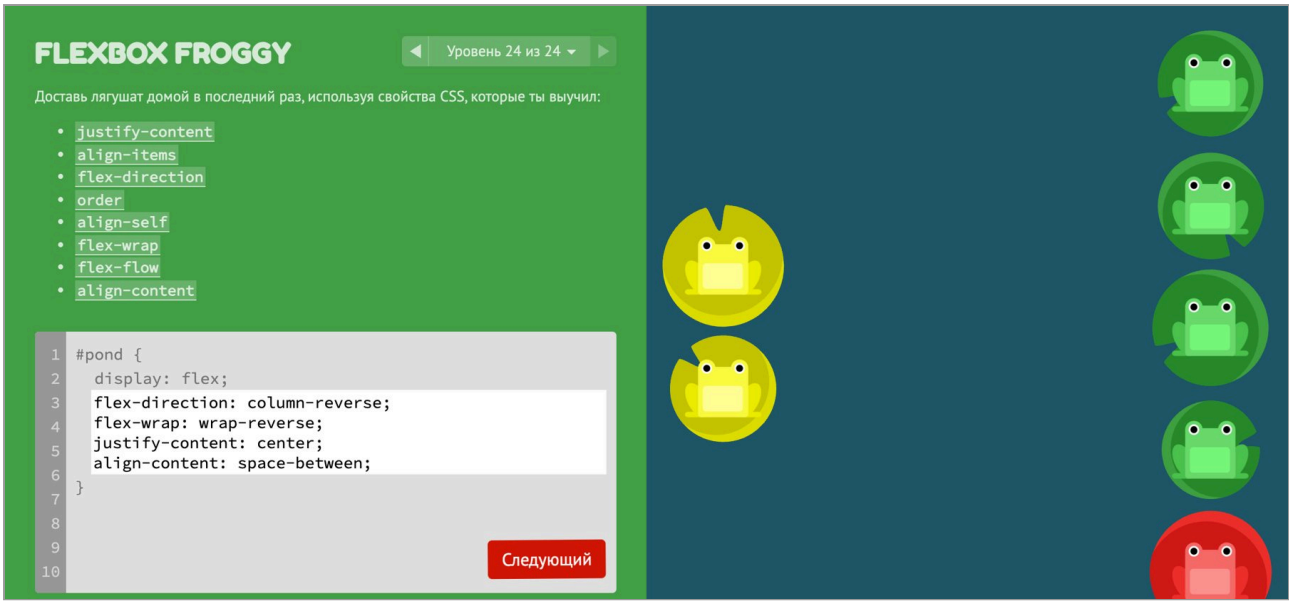


Рисунок 1 — решение 24-го (последнего) уровня Flexbox Froggy



Рисунок 2 — экран завершения Flexbox Froggy: все 24 уровня отмечены как пройденные

2. CSS Grid Garden

В игре нужно полить морковь (`#water`) и обработать сорняки (`#poison`), размещая участки в сетке грядки `#garden` (`display: grid`, по умолчанию 5×5). По ходу игры изучены:

- `grid-column-start`, `grid-column-end`, `grid-row-start`, `grid-row-end` задают размещение элемента по линиям сетки. Отрицательные номера отсчитывают линии с конца, а `span N` растягивает элемент на N ячеек;
- сокращения `grid-column` и `grid-row` в формате `начало / конец`, а также `grid-area` в формате `row-start / column-start / row-end / column-end`;
- `order` меняет порядок автоматически размещаемых элементов;
- `grid-template-columns` и `grid-template-rows` задают размеры дорожек в `%`, `px`, `em` и долях свободного места `fr`. Для повторяющихся дорожек используется функция `repeat()`;
- сокращение `grid-template: строки / столбцы`.

Все 28 уровней пройдены. Решения приведены в таблице 2.

Таблица 2. Решения уровней CSS Grid Garden

№	Тема	Селектор	Решение
1	grid-column-start	`#water`	`grid-column-start: 3;`
2	grid-column-start	`#poison`	`grid-column-start: 5;`
3	grid-column-end	`#water`	`grid-column-end: 4;`
4	grid-column-end	`#water`	`grid-column-end: 2;`
5	grid-column-end (отриц.)	`#water`	`grid-column-end: -2;`
6	grid-column-start (отриц.)	`#poison`	`grid-column-start: -3;`
7	span	`#water`	`grid-column-end: span 2;`
8	span	`#water`	`grid-column-end: span 5;`
9	span	`#water`	`grid-column-start: span 3;`
10	grid-column	`#water`	`grid-column: 4 / 6;`
11	grid-column	`#water`	`grid-column: 2 / 5;`
12	grid-row-start	`#water`	`grid-row-start: 3;`
13	grid-row	`#water`	`grid-row: 3 / 6;`
14	grid-column + grid-row	`#poison`	`grid-column: 2;` `grid-row: 5;`
15	grid-column + grid-row	`#water`	`grid-column: 2 / 6;` `grid-row: 1 / 6;`
16	grid-area	`#water`	`grid-area: 1 / 2 / 4 / 6;`
17	grid-area	`#water-2`	`grid-area: 2 / 3 / 5 / 6;`
18	order	`#poison`	`order: 2;`
19	order	`.poison`	`order: -1;`

20	grid-template-columns	#garden	`grid-template-columns: 50% 50%;`
21	repeat()	#garden	`grid-template-columns: repeat(8, 12.5%);`
22	px, em, %	#garden	`grid-template-columns: 100px 3em 40%;`
23	fr	#garden	`grid-template-columns: 1fr 5fr;`
24	fr + px	#garden	`grid-template-columns: 50px 1fr 1fr 1fr 50px;`
25	fr + px	#garden	`grid-template-columns: 75px 3fr 2fr;`
26	grid-template-rows	#garden	`grid-template-rows: 50px 0 0 0 1fr;`
27	grid-template	#garden	`grid-template: 60% 40% / 200px 1fr;`
28	grid-template	#garden	`grid-template: 1fr 50px / 20% 80%;`



Рисунок 3 — решение 28-го (последнего) уровня CSS Grid Garden

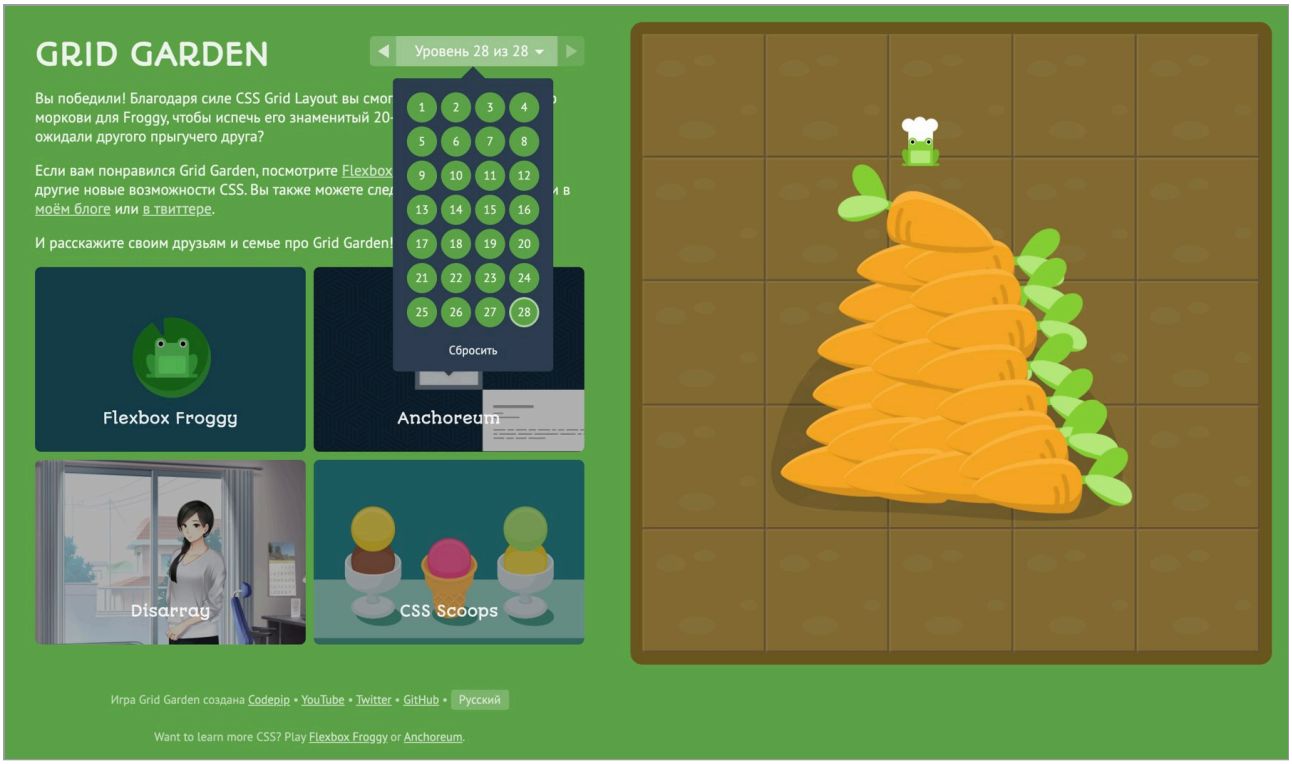


Рисунок 4 — экран завершения CSS Grid Garden: все 28 уровней отмечены как пройденные

3. Learn Git Branching

Learn Git Branching визуализирует репозиторий git. Команды вводятся в терминал, а дерево коммитов на экране меняется сразу после каждой команды. Цель уровня — привести репозиторий к заданному целевому состоянию. Уровни разделены на две вкладки: «Основы» и «Удаленные репозитории».

На вкладке «Основы» пройдены первые четыре раздела: «Введение», «Едем дальше», «Перемещаем труды туда-сюда» и «Сборная солянка» (17 уровней). На вкладке «Удаленные репозитории» пройдены все уровни обоих разделов (16 уровней). Всего пройдено 33 уровня (рисунки 5 и 6). Раздел «Введение» состоит ровно из четырёх уровней, поэтому требование «первые 4 урока» выполнено при любом прочтении.

В ходе прохождения отработаны:

- коммиты, ветки и переключение между ними: `git commit`, `git branch`, `git checkout` / `git checkout -b`;
- объединение изменений: `git merge` и `git rebase`. Merge создаёт коммит слияния с двумя родителями, rebase переносит коммиты поверх другой ветки и делает историю линейной;
- перемещение по истории: detached HEAD, относительные ссылки `^` и `~N`, перемещение веток командой `git branch -f`;
- отмена изменений: `git reset` для локальной истории и `git revert` для уже опубликованной;
- перенос отдельных коммитов: `git cherry-pick` и интерактивный `git rebase -i`, в котором коммиты можно переставлять и исключать (рисунок 7);
- индекс: `git add`, `git restore` и `git restore --staged`. Метки и описание коммитов: `git tag`, `git describe`;
- работа с удалённым репозиторием: `git clone`, удалённые ветки вида `o/main`, `git fetch`, `git pull` (fetch + merge), `git pull --rebase`, `git push`;
- решение конфликтов истории при расхождении с origin. Когда ветка main защищена, изменения публикуются через отдельную feature-ветку;
- отслеживание удалённых веток (`git checkout -b side o/main`) и аргументы `push` / `fetch` / `pull` в формате `<источник>:<назначение>`. Пустой источник (`git push origin :foo`) удаляет ветку на удалённом репозитории, а `git fetch origin :bar` создаёт новую локальную ветку.

Команды, которыми решены уровни, приведены в таблице 3.

Таблица 3. Решения уровней Learn Git Branching

№	Уровень	Команды
Основы → 1. Введение		
1	Знакомство с Git Commit	`git commit` `git commit`
2	Ветвление в Git	`git branch bugFix` `git checkout bugFix`

3	Слияния веток в Git	`git checkout -b bugFix`
		`git commit`
		`git checkout main`
		`git commit`
		`git merge bugFix`

4	Введение в rebase	`git checkout -b bugFix`
		`git commit`
		`git checkout main`
		`git commit`
		`git checkout bugFix`
		`git rebase main`

Основы → 2. Едем дальше		
1	Теряем голову, или detached HEAD	`git checkout C4`
2	Относительные ссылки (^)	`git checkout bugFix^`

3	Относительные ссылки №2	`git branch -f main C6`
		`git checkout HEAD~1`
		`git branch -f bugFix HEAD~1`

4	Отмена изменений в Git	`git reset HEAD~1`
		`git checkout pushed`
		`git revert HEAD`

Основы → 3. Перемещаем труды туда-сюда		
1	Введение в Cherry-pick	`git cherry-pick C3 C4 C7`
2	Введение в интерактивный Rebase	`git rebase -i overHere` *в диалоге: C2 исключён, порядок C3, C5, C4*

3	Область подготовленных файлов (Индекс)	`git add app.js`
		`git commit`
		`git add styles.css`
		`git commit`

4	Отмена изменений с помощью git restore	`git restore --staged secret.env`
		`git restore experiment.js`
		`git commit`

Основы → 4. Сборная солянка		

1	Выберем один коммит.	`git rebase -i main`
		в диалоге оставлен только C4
		`git rebase bugFix main`

2	Жонглируем коммитами	`git rebase -i HEAD~2` *(порядок C3, C2)*
		`git commit --amend`
		`git rebase -i HEAD~2` *(порядок C2'', C3')*
		`git rebase caption main`

3	Жонглируем коммитами №2	`git checkout main`
		`git cherry-pick C2`
		`git commit --amend`
		`git cherry-pick C3`

4	git tag	`git tag v1 side~1`
		`git tag v0 main~2`
		`git checkout v1`

5	Git describe	`git commit`
Удаленные репозитории → Push & Pull — удалённые репозитории в Git!		
1	Введение в клонирование	`git clone`

2	Удалённые ветки	`git commit`
		`git checkout o/main`
		`git commit`

3	Git fetch	`git fetch`
4	Git pull	`git pull`

5	Коллективная работа	`git clone`
		`git fakeTeamwork 2`
		`git commit`
		`git pull`

6	Git push	`git commit`
		`git commit`
		`git push`

7	Расхождение в истории	`git clone`
		`git fakeTeamwork`
		`git commit`
		`git pull --rebase`
		`git push`

8	Заблокированная ветвь main	`git branch -f main o/main`
		`git checkout -b feature C2`
		`git push origin feature`

Удаленные репозитории → Через origin — к звёздам. Продвинутое использование Git Remotes		

1	Push Мастер!	`git fetch`
		`git rebase o/main side1`
		`git rebase side1 side2`
		`git rebase side2 side3`
		`git rebase side3 main`
		`git push`

2	Слияние с удалённым репозиторием	`git checkout main`
		`git pull`
		`git merge side1`
		`git merge side2`
		`git merge side3`
		`git push`

3	Слежка за удалённым репозиторием	`git checkout -b side o/main`
		`git commit`
		`git pull --rebase`
		`git push`

4	Аргументы git push	`git push origin main` `git push origin foo`
5	Аргументы для push -- расширенная версия!	`git push origin main^:foo` `git push origin foo:main`

6	Аргументы для fetch	`git fetch origin C3:foo`
		`git fetch origin C6:main`
		`git checkout foo`
		`git merge main`

7	Пустой источник	`git push origin :foo` `git fetch origin :bar`
8	Аргументы для pull	`git pull origin C3:foo` `git pull origin C2:side`

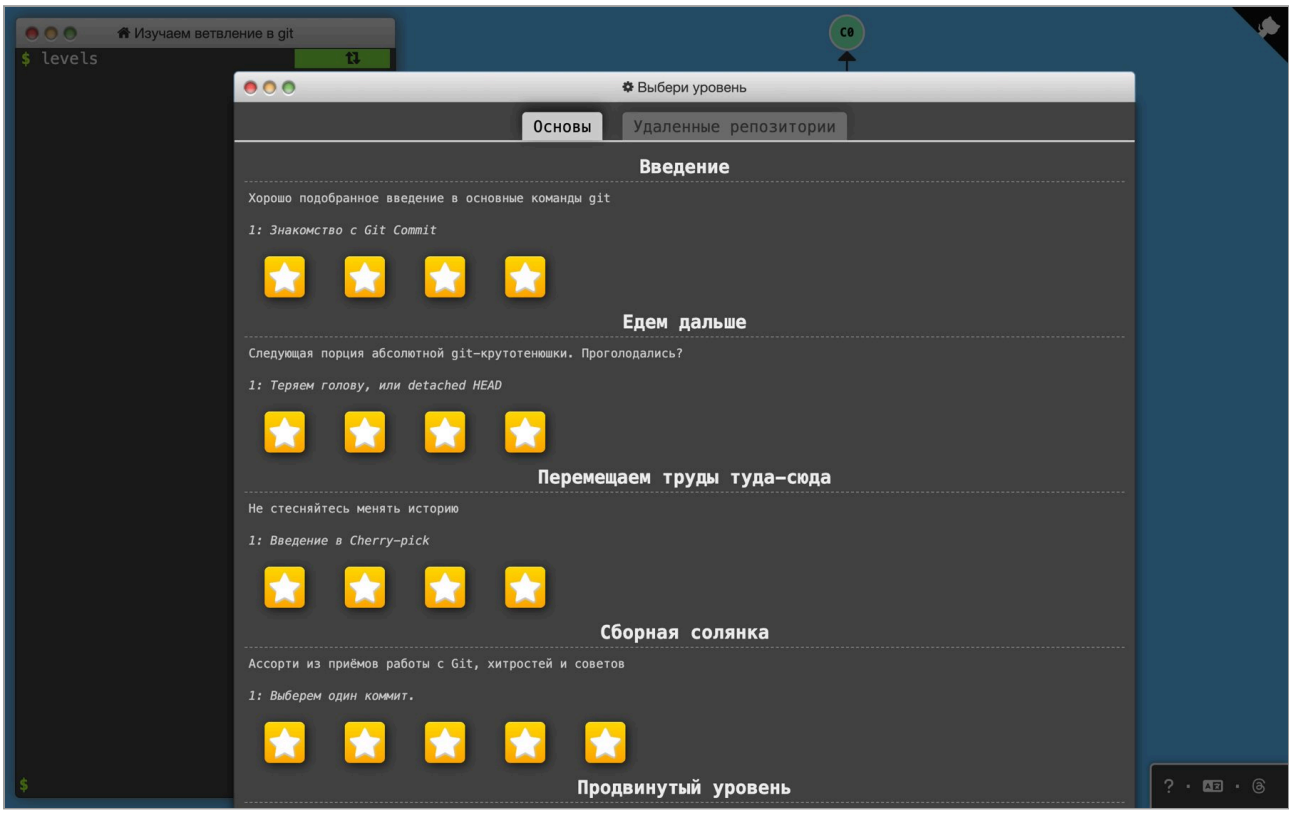


Рисунок 5 — вкладка «Основы»: все уровни разделов 1–4 отмечены как пройденные

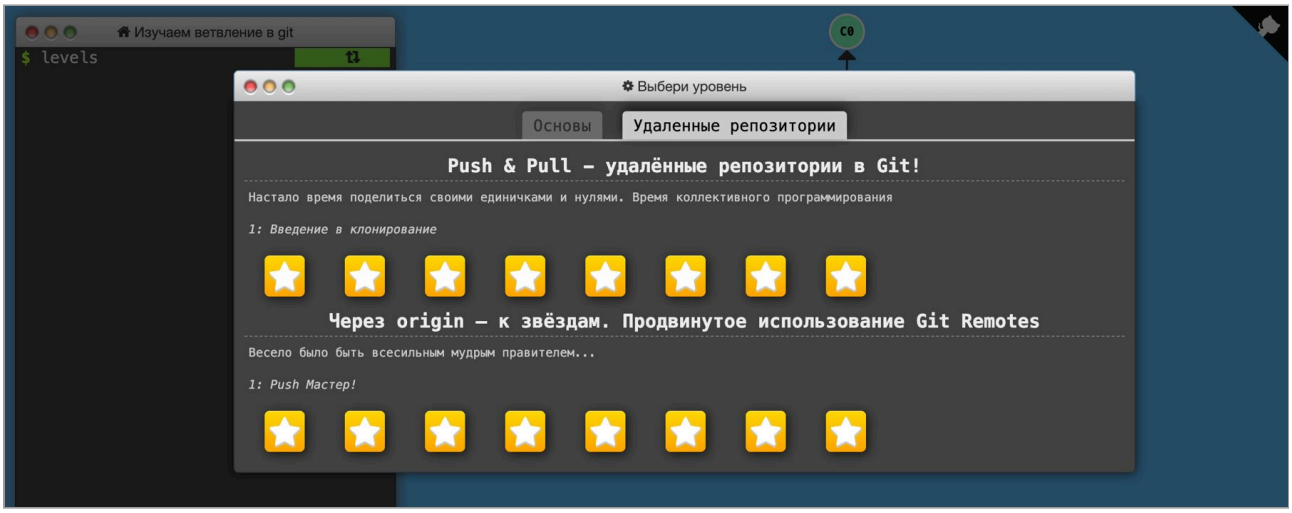


Рисунок 6 — вкладка «Удаленные репозитории»: все 16 уровней отмечены как пройденные

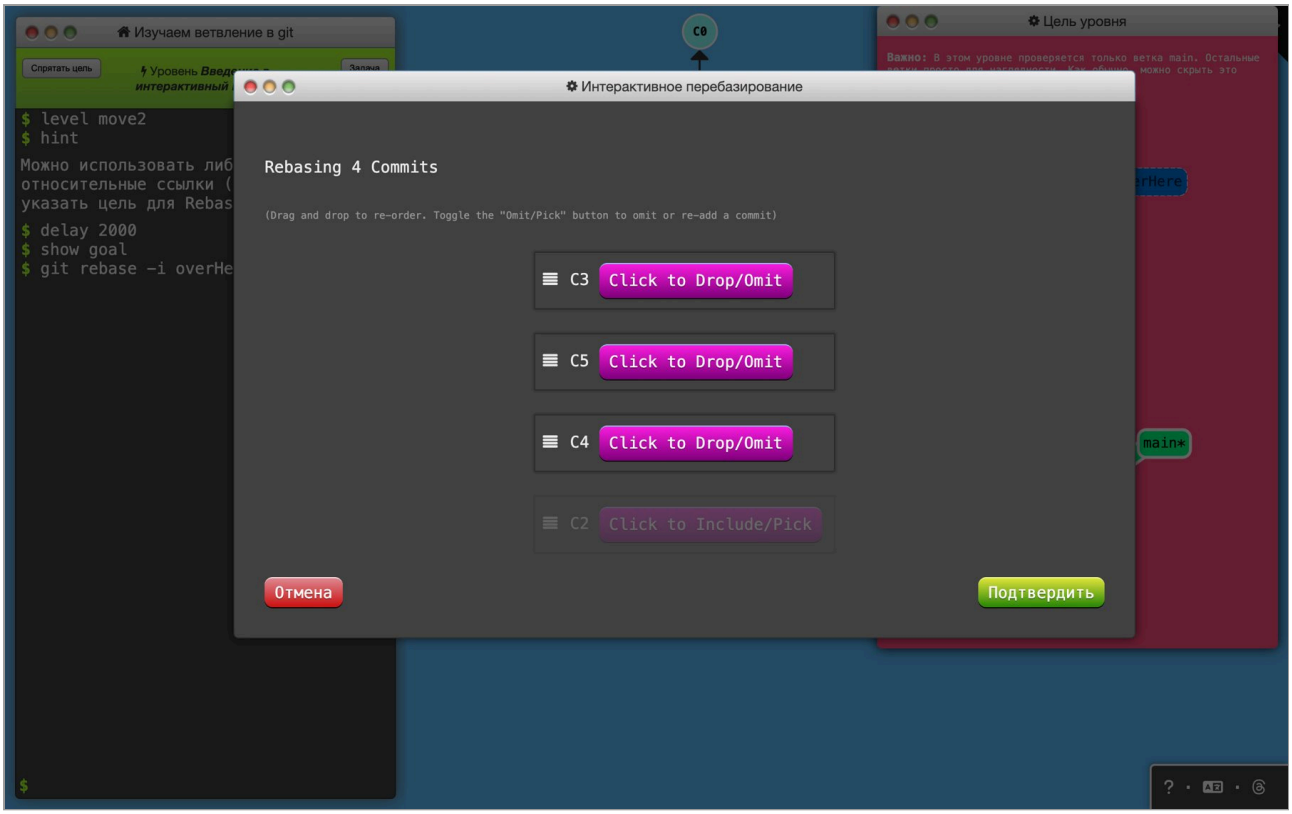


Рисунок 7 — интерактивный rebase в уровне «Введение в интерактивный Rebase»: коммит C2 исключён, остальные переставлены в порядке C3, C5, C4

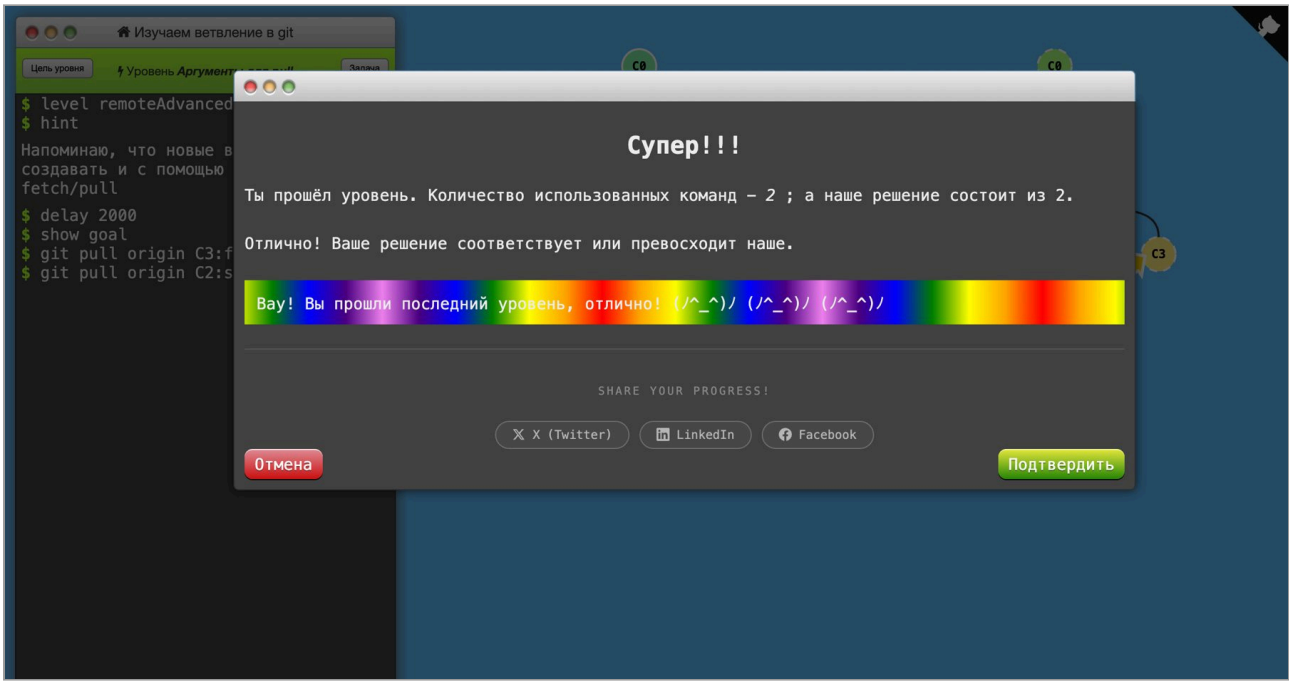


Рисунок 8 — прохождение последнего уровня «Аргументы для pull» на вкладке «Удаленные репозитории»

Вывод

В ходе работы пройдены все 24 уровня Flexbox Froggy и все 28 уровней CSS Grid Garden. Получены практические навыки одномерной раскладки на Flexbox: выравнивание по главной и поперечной осям, смена направления, перенос строк, управление порядком и выравниванием отдельных элементов. Освоена двумерная раскладка на CSS Grid: размещение элементов по линиям сетки, растягивание через `span`, сокращённые свойства `grid-column`, `grid-row` и `grid-area`, описание сетки через `grid-template-*` с единицами `fr` и функцией `repeat()`. Flexbox удобен для выравнивания элементов в одном направлении, а Grid подходит для макетов, где важны и строки, и столбцы.

В Learn Git Branching пройдено 33 уровня: первые четыре раздела вкладки «Основы» и все уровни по удалённым репозиториям. Закреплены работа с коммитами и ветками, merge и rebase, перемещение по истории, cherry-pick и интерактивный rebase, а также синхронизация с удалённым репозиторием через `clone`, `fetch`, `pull` и `push`, включая их аргументы. Эти навыки нужны для командной разработки и для сдачи работ в этом курсе через Pull Request.